

Loop Engineering

把「做不完、信不過、會中斷、會卡死」的任務，交給一個會自己收斂的迴圈

例會分享 · loop-agent 專案

這個專案在解什麼問題？

一個 AI agent，一次性地做一件「大任務」時，會遇到六個現實問題：

#	問題	白話
1	單次做不完	超過 context / 時間上限
2	單次信不過	會漏看、腦補、判斷錯
3	會中斷	斷線、換模型、context 用完
4	會把檔案改壞	整檔重寫成空白 / 亂碼
5	會卡死	改 A 壞 B，原地打轉
6	不知道何時該停	停止條件不可判讀

目標：對齊目標、可接手、可驗證、可恢復、可收斂、不卡死、能自動停止。

核心心法：一句話講完

把一個大任務拆成

「一份可持續推進的主控文件」 + 「反覆被觸發的無狀態 agent」
+ 「多層收斂驗證」 + 「震盪偵測與升級」 + 「Git 安全網」

讓任何 agent 中斷、換 context、換模型，都能只靠這份文件接手，
做到對齊目標且全部收斂才停。

Loop (迴圈) 負責「反覆推進」；

Harness (外部束具) 負責「讓推進這件事本身安全、可控、不失控」。

Harness 是什麼？—— agent 不是迴圈本身

 最關鍵的心智模型翻轉：

- Agent **不是**那個一直跑的東西
- Agent 是**外部迴圈**呼叫一次的函式——被叫醒、做一件事、交回控制權、結束 process
- 真正「一直跑」的是外部的 **harness** (`loop.py` / `plan_loop.py`)

```
for round in 1..max_rounds:  
    harness: 前置檢查 → 選模型 → 啟動全新 agent (subprocess)  
    agent:   做「唯一一件事」→ 落地 → 更新狀態 → 立刻結束  
    harness: 補 commit → 偵測震盪 → 升級模型 → sleep → 回到迴圈頂
```

穩定交棒 > 一次衝完。這就是「loop engineering」與「叫 agent 做完一整包事」的根本差異。

三段式生命週期（兩個人類決策點）

- ① 需求討論 → REQUIREMENTS.md
↓  人類確認需求 (gate #1)
- ② 規劃迴圈 plan_loop.py → state.json + phases/ (跑到收斂)
↓  review 規劃書 (gate #2, 僅 gated 模式)
- ③ 執行迴圈 loop.py → 反覆喚醒全新 agent, 做到收斂
↑ 卡住 / 震盪 → 自動升級模型 → 真卡死才停下交人

- **gated (預設)** : 規劃收斂 → 停下交人 review → 人類放行才執行
- **auto** : 規劃收斂後直接接執行，一路衝到收斂或卡死

人類只需要在意兩件事：需求對不對、規劃書合不合理。其餘交給迴圈。

Harness 每輪疊了幾層防護？

執行迴圈 `loop.py` 每一輪，在呼叫 agent 之前後都包了一層機器防線：

- ① log rotate / 鎖心跳
- ② [G0] 主控檔毀損自檢 → 壞了自動 git checkout 還原
- ③ [Git Review Gate] 獨立 agent 審上一輪 diff → PASS / REVERT / FATAL
- ④ 停止判定 (正常完成 / human_required)
- ⑤ 依 stuck_level 選模型 (fast / normal / thinking)
- ⑥ subprocess 啟動全新無狀態 agent (watchdog: round timeout / idle timeout)
- ⑦ git_guard: 補 autocommit (本輪還原點)
- ⑧ 震盪 / 卡住偵測 → 達門檻升級 stuck_level
- ⑨ sleep(interval) → 回到迴圈頂

agent 行為層之外，還有一整套機器層的安全網——這才是 harness 的價值所在。

為什麼「單次信不過」：三層收斂

不相信 agent 一次做對，用**獨立重推 + 比對**逼近真相：

層級	防什麼	收斂判準
單任務收斂	單支結果腦補 / 判斷錯	新 context 獨立重推，連續 N 次一致
集合穩定性收斂	大範圍漏看	每輪重新列舉，連續 N 輪集合不再增減
全量驗證收斂	跨模組不一致	全量驗證連續 N 次 PASS

收斂鐵則：

- 先**獨立重推**，再跟既有比對——不可先看舊結果說「看起來沒問題」（橡皮圖章）
- 任何一次發現差異 → 修正 + **收斂計數歸零重數**
- **無證據檔的那一輪不算數**（要留下可稽核紀錄）

痛點實例：任務切太粗，怎麼被磨到剛好

情境：規劃時 agent 把一個功能寫成一個大任務，結果一輪做不完、也沒辦法被單獨驗證。

Harness 不是靠「一次猜對大小」解決，是靠迴圈磨出來：

- ① 規劃 agent 拆出一批任務（可能還是切太粗）
- ② 獨立 review agent（全新 context）逐一檢查：
這塊「一輪做得完」嗎？「能被單獨驗證」嗎？
- ③ 不合格 → 打回、要求把這塊再往下拆一層（回饋，不是整批重寫）
- ④ 下一輪重新「列舉」整批任務、跟上一輪比對
- ⑤ 連續 N 輪：任務清單不再增減、每塊都通過 review → 判定「集合穩定」
- ⑥ 穩定之後才進人類最終 gate：人類只看「拆得對不對」，
程式只保證「拆得穩不穩」——兩件事故意分開

這其實是上一頁「集合穩定性收斂」用在「任務切分」本身：

切 → review 打回 → 再切 → 再列舉比對，磨好幾輪才收斂，不是一次到位。

為什麼「會卡死」有解：震盪偵測 + 升級階梯

三種卡法：不收斂（同問題修不好）／震盪 A  B（原地繞）／無活動（連續沒提交）

三模型二維調度（同一組模型，一魚兩吃）：

- 角色維：拆解用 thinking、review 用 normal、執行用 fast
- 升級維：卡住時 fast → normal → thinking → 交人

Lv0 角色預設模型 → 偵測卡住 → Lv1 升級模型（注入「先判根因」）
→ 仍卡 M 輪 → Lv2 人類接管：開 Issue + FROZEN + human_required=true

換模型只治「能力不足」，治不了「規格矛盾」。

逃生門（FROZEN + 交人）比任何自動修復都重要。

授權紅線：機器判事實，人判價值

程式只被授權做「可機器判讀」的決定：收斂計數、輪數、深度、回流次數...

凡牽涉**價值判斷**，一律邊界外、撞到就停、交人：

-   「規格本身有沒有矛盾」不該由程式判
-   「這棵任務樹拆得對不對」不該由程式判（穩定 ≠ 正確）
-   「要不要放寬邊界繼續」絕不該由程式判

一個能自己放寬的邊界，就不是邊界。

硬 BREAKER (max_depth / max_leaf_reflow / growth_stall_rounds) 撞線只會凍結交人，程式不准自我放寬。

Git 是安全網，不是版本控制而已

- 一輪一 **commit**：每個 commit 都是乾淨的還原點
- 開機自檢：下一輪先驗證主控檔健康，被寫壞就 `git checkout HEAD --` 救回
- 寫檔守則：局部編輯優先、寫完讀回確認
- 安全紅線：禁止 `reset --hard` / `clean -fd` / `push --force`，只允許逐檔還原

因為 agent 偶爾會把檔案整支重寫成空白——harness 假設「agent 會犯這種錯」，用 git 而不是信任把這個風險兜住。

弱模型會怎麼鑽漏洞（規則不夠硬時）

把自己想成「一個被反覆呼叫、想用最低成本過關」的模型，規則寫得鬆就會被這樣鑽：

弱點	弱模型會怎麼做
一輪偷做多個	挑到一個任務後「順便」多做兩三個，看似有效率、實則破壞可交棒性
收斂造假	沒有重新推導，直接看舊結果說「我看過了、一致」就蓋章通過
逃生閥濫用	卡住的任務隨手標成 <code>FROZEN</code> / <code>NON_BLOCKING</code> ，繞過真正該解決的問題
軟性措辭鑽空子	規則寫「盡量 / 必要時 / 視情況」，模型挑對自己最省事的解讀
生命週期不停機	做完一輪後自己接著跑下一輪，不物理停機交棒
自我放寬邊界	卡住時「我覺得可以放寬門檻繼續」，自行鬆綁硬限制

這不是模型「壞」，是任何會被反覆呼叫、想用最低成本收斂的角色都會這樣——所以不能只靠 prompt 拜託，要靠機器擋。

因此：狀態不准 agent 自己改，一律經過 `state.py`

Agent 嚴禁直接編輯 `state.json` ；所有變更都得走 `state.py` CLI，CLI 內建硬校驗，正好擋下上一頁的鑽法：

鑽法	<code>state.py</code> 怎麼擋
任務直接從 <code>TODO</code> 跳到 <code>CONVERGED</code>	只允許合法單步轉移（ <code>TODO→DRAFTED→CONVERGED ...</code> ），跳級直接拒絕
收斂計數用嘴巴講講就 +1	改 <code>CONVERGED</code> 前檢查 <code>conv</code> 是否真的達門檻； <code>conv++</code> 綁「Phase+Commit」簽章，沒有新 <code>commit</code> 連按兩次無效
一輪偷做好幾個任務	同一個 <code>run_id + round</code> 只允許一次「實質推進」，第二次直接拒絕
卡住後把 <code>human_required</code> 自己改回 <code>false</code>	一旦為 <code>true</code> ， <code>set</code> 改不掉，只能走人類的 <code>resume</code> 管道解除
隨便塞一個沒宣告過的欄位	鍵值白名單，非白名單 <code>key</code> 一律拒絕寫入

讓 Harness 自我增長：用真實執行痕跡持續變硬

前面講的是「單一任務」怎麼被迴圈做完；這裡多一層問，**框架本身要不要也跟著變強？**

每個專案跑完，都留下「這輪卡在哪、震盪了幾次、換了幾次模型」的紀錄（trace）。把多個專案的紀錄收集起來，就能看出「哪裡是這個框架真正、反覆讓人踩坑的地方」：

收集多個專案的 trace（純讀、離線）



分析 agent 找出「跨專案重複出現」的痛點（附證據：出處、次數）



產出帶證據的修改提案（給 rules / prompts / config 的草稿 diff）



  人類 review + 手動合併進框架

硬約束：

- 只有 $\geq K$ 個不同專案都踩到同一種痛，才算「該修框架」——避免拿單一專案的意外情況，去改壞所有人共用的規則
- 分析 agent 只能提案，絕不自己合併——合併永遠是人類的權力

九大原則，一頁總覽

1. 目標是一等公民——需求 → 任務 → 驗證可追溯
2. 文件即狀態——換一個新 agent 要能只靠文件接手
3. 單一進入點 + 分檔——`state.json` 每輪必讀，分檔按需讀
4. 分階段 + 階段門——前一階段全收斂才放行下一階段
5. 每輪只做一件事——硬規則，做完立刻結束 process
6. 不猜測——不確定就開 Issue，不腦補
7. 可追溯——結論附來源，產出標明實作自哪份規格
8. **Git 是安全網**——每輪一個還原點
9. 卡死要有逃生門——升級到人類，價值判斷永遠交人

其他踩過的坑（框架設計面，節錄）

反模式	後果	正解
狀態只存在 agent context	中斷就重來	狀態寫進文件（ <code>state.json</code> + <code>state.py</code> ）
停止條件設「Open Issue = 0」	外部依賴永遠死結	改成「無 BLOCKING Issue」
agent 整檔重寫	寫成空白	局部編輯 + 讀回確認 + git 還原

為什麼這件事值得做成「框架」而不是一次性腳本

- 這六個問題（做不完/信不過/會中斷/會改壞/會卡死/不知道停）在任何大型 agent 任務上都會重現
- 一次性寫死在某個專案的腳本裡 → 下個專案重踩一次坑
- 抽成**唯讀、共享的框架**（本專案），下游 code repo 只落 `.loop/<name>/` :
 - 框架本身不被寫入，只被讀取
 - 一次把「防護」寫好，所有專案都受益
 - 甚至能讓框架自己靠真實執行資料持續變硬

總結：Loop Engineering 的精神

把「不可信、做不完、會中斷、會卡死」的大任務，
轉成「一份會自己記錄進度與信心、能自我修正、卡住會升級、
能自動判斷何時收斂或求救」的活文件；
agent 只是反覆來推它一把的手，**harness** 才是讓這件事能放心自動跑的骨架。

對齊目標、穩定、可追溯、不猜測、能還原、卡死有逃生門——比跑得快更重要。

Q&A